

Project Report

Project Name: PawDuty

Reference ID: SDZ0CU9I

1. Executive Summary

PawDuty is a centralized pet care coordination app that turns scattered group chats, sticky notes, and verbal reminders into a single source of truth for multi-person households caring for pets. It combines shared chore management, vaccination/medication tracking, and an AI-powered camera scan that reads a cat's body temperature from a thermal image to flag possible early signs of illness by helping households avoid missed care and catch potential problems earlier, without replacing a vet's judgment.

2. Project Overview

2a. Why you're building and what you're building

Cat care in shared households is rarely a solo job, but it's almost always tracked like one through memory, verbal hand-offs, or a group chat message that gets buried within minutes. This informality creates real, recurring problems like a cat gets fed twice because two roommates each assumed the other hadn't done it yet, a litter box goes uncleaned for days because no one explicitly owned it, or a vaccination deadline slips because it lived in only one person's head. Perhaps most importantly, subtle signs of illness like a dull coat, a slight limp, a change in eye clarity often go unnoticed because no single caretaker sees the cat consistently enough, and observations aren't shared across the household. PawDuty exists to close that visibility gap to make pet care a shared, trackable responsibility instead of a set of individual assumptions, and to give households an early, low-effort way to notice when something might be wrong including a camera-based scan that reads a cat's temperature from a thermal image to help flag a possible health concern before it escalates.

2b. How it relates to the theme

A cat's world domination starts at home with reliably fed, medicated, and healthy cats, and humans who feel confident and unstressed about providing that care. PawDuty makes it dramatically easier for any household, regardless of size or organization skill, to give consistently excellent care. Fewer missed feedings, fewer missed vaccinations, and earlier catches on health issues means happier, healthier cats and less caretaker burnout which is exactly the kind of wholesome, compounding impact the theme calls for: better outcomes for cats, and for the humans who serve them.

2c. Target Audience

- **Household Organizers** — the person who sets up chores, manages the pet profile, and wants visibility into whether things actually get done.
- **Co-Caretakers** — roommates, partners, or family members who share day-to-day duties and need a simple, low-friction way to see what's expected of them and check it off.

3. Key Features

Task management (CRUD, backed by task-api)

- Create tasks with category (Vaccination / Medicine / Grooming / Other), pet, assignee, date, optional time, repeat (Once / Daily / Weekly / Monthly), and notes.
- Edit any task (tap a card → prefilled form → save = PATCH).
- Delete a task (trash icon → confirm).
- Toggle done from the list — optimistic UI with revert on failure.
- Server generates task ids and derives assignee initials from the name.

Home

- Greeting + Daily / Weekly / Monthly segmented filter.
- Loading spinner while tasks load; an error banner with Retry if a service is unreachable (API is the source of truth — no stale cache).

Progress

- Month calendar with per-day multi-dot markers (done vs pending).
- Completion pie chart (Done vs Pending) for the selected period.
- Total / Done / Pending stat tiles.

AI thermal health scan

- Full-screen camera viewfinder with a scan frame; capture → classify.
- Result card shows HEALTHY / NEEDS ATTENTION, a plain-language explanation, and the model's confidence.
- "Add as Task" prefills a follow-up (e.g. a vet check-up) on the Add screen.
- Graceful errors: unreachable service or "no cat body detected" (422) surface a message.

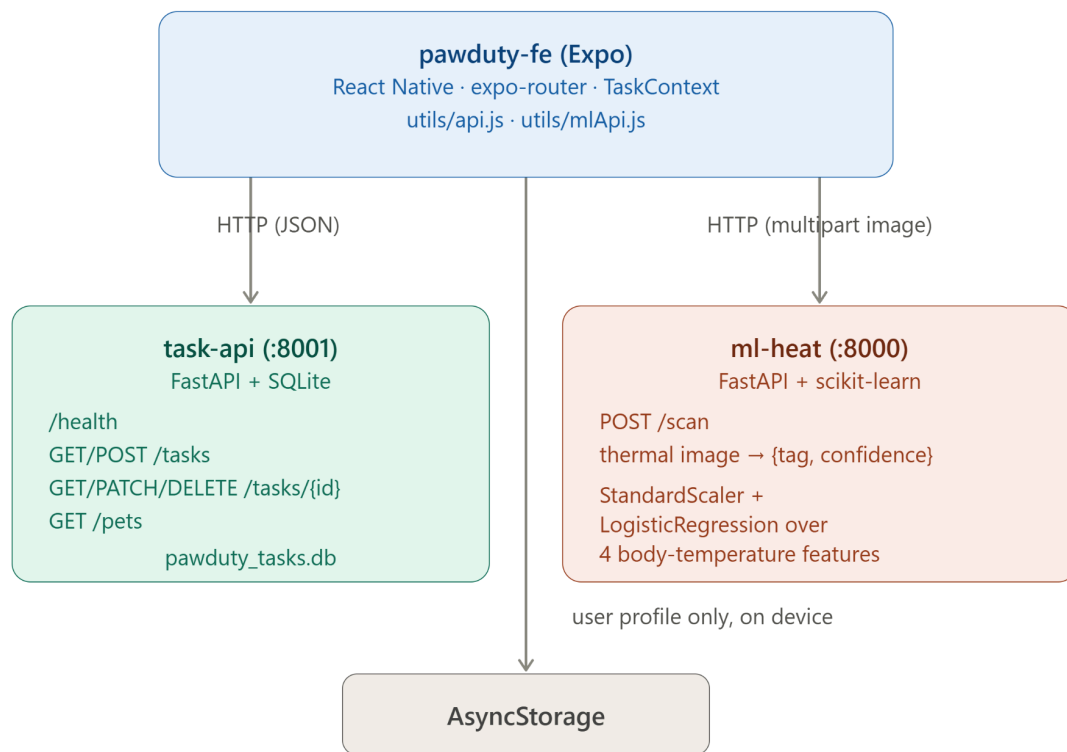
Profile

- Editable display name (persisted to AsyncStorage).
- Pets list (name, breed, colour) from the backend.
- Total-tasks / done-this-month stats; notification & dark-mode setting placeholders.

4. Technology Stack

Layer	Technology
Mobile app	React Native 0.86, React 19, Expo SDK 57 (managed), Expo Router (file-based tabs)
App state	React Context + hooks; @react-native-async-storage/async-storage (user profile only)
Networking	fetch (single utils/api.js + utils/mlApi.js boundaries), base URL auto-derived via expo-constants
UI / device	@expo/vector-icons (Ionicons), expo-camera, @react-native-community/datetimepicker, react-native-calendars, react-native-chart-kit, react-native-svg
task-api	Python 3, FastAPI, Pydantic v2, stdlib sqlite3 (no ORM), Uvicorn
ml-heat service	Python 3, FastAPI, scikit-learn (StandardScaler + LogisticRegression), OpenCV (opencv-python-headless), NumPy, pytesseract + system Tesseract OCR, joblib
Testing	Jest + jest-expo + @testing-library/react-native (app); pytest + httpx TestClient (both services)

5. Technical Architecture



Data flow

- Tasks & pets — the source of truth is task-api (FastAPI + SQLite). TaskContext loads tasks and pets on mount, and every create/edit/toggle/delete is an HTTP call to task-api. On first run the service seeds 6 demo tasks + 2 pets.
- User profile — stored on-device in AsyncStorage (there is no user endpoint).
- AI thermal scan — the AI tab captures a photo with expo-camera and POSTs it to the ml-heat service /scan, which returns { tag: "healthy" | "unhealthy", confidence }. The result can be turned into a follow-up task.
- Host discovery — both API base URLs are derived from the Expo dev host at runtime (expo-constants), so a phone on the same Wi-Fi reaches the services with no manual IP. Each has an OVERRIDE constant for tunnel/production use.

6. Testing Matrix

	Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
1	First-run seed	Fresh DB → GET /tasks, GET /pets	6 tasks, 2 pets	count=6, count=2	Pass
2	Create task	POST /tasks with no id, assignee: {name:" Al Bee"}	201, server id, initials derived, done:false	HTTP 201, id=dda46846..., initials=AB	Pass
3	Read + Daily/Weekly/Monthly filter	Select each segment on Home	Only tasks in today / ISO week / month	dateFilters 4/4 tests pass; daily = today only	Pass
4	Toggle done (optimistic)	Tap checkbox → PATCH {done:true}	Flips immediately, reconciles with server; reverts on failure	PATCH → done:true; TaskContext revert test passes	Pass
5	Edit task	Tap card → change fields → save → PATCH	Only changed fields updated; initials re-derived	title="Renamed via edit", initials=CD	Pass
6	Delete task (optimistic)	Trash → confirm → DELETE, then GET	204, task gone (404); count restored	204 → 404; count 6→7→6	Pass
7	Input validation	POST blank title / bad category	422 both	422, 422	Pass
8	Offline error + retry (Home)	task-api down → open Home → tap Retry	Error banner + "Retry" re-calls API	Banner shown; listTasks called ×2 on retry (HomeScreen test)	Pass

	Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
9	AI scan — API contract	POST /scan with a thermal JPEG; and a blank frame	200 {tag, confidence}; 422 when no body	sick fixture → {"unhealthy",0.68}; blank → 422; service 13/13	Pass
10	AI scan — UI (result / error)	Mock scan resolve/reject in app	Result card (HEALTHY/ NEEDS ATTENTION) / error text	AIScreen 4/4 tests pass	Pass
11	AI scan — model accuracy	Classify labelled fixtures with the real model	~76.5% (research baseline; imperfect)	sick→unhealthy  ; 2 healthy fixtures → unhealthy ✗ — consistent with 76.5% CV	 Baseline (76.5% ± 4.5%)
12	Progress view	Open Progress	Segments + Total/Done/Pending + calendar + chart	ProgressScreen 2/2 tests pass	Pass
13	Profile edit persists	Edit name → save	Name updates + written to AsyncStorage	TaskContext updateUser test passes	Pass
14	Full regression	Run all three suites	0 failures	FE 54 + task-api 39 + ml-heat 13 = 106 passed	Pass

7. Future Improvements

- **Household streaks or a light completion-rate view** — not gamified to the point of being gimmicky, but enough visibility to make consistent care feel noticed.
- **Home-screen widget** — a glanceable "today's duties" widget so people don't even have to open the app to see what's left.
- **Household permission refinement** — clearer rules for who can edit or delete chores created by others, and what happens to shared records when a member leaves the household.
- **Multi-pet and multi-species groundwork** — extending the data model to more comfortably support multiple cats per household now, with an eye toward dogs in a later phase.

8. Tools You Used

- Claude Code
- Excalidraw
- VS Code
- Railway

9. Learnings & Takeaways

- **The hardest part of a coordination product is the trust layer underneath the features, not the features themselves.** — A chore-tracking app is straightforward to build; a chore-tracking app people actually rely on instead of falling back to a group chat requires getting reminder timing, completion visibility, and permission rules exactly right — get any of those wrong and people quietly stop using it.
- **Multi-user products need their edge cases designed early, not patched in later.** — Working through household permissions and notification fatigue showed us that decisions like who can delete someone else's chore, or what happens to shared pet records when a roommate moves out, quietly shape the entire trust model of the app.
- **Trend data is often more useful than a single scan.** — A one-off confidence score is easy to misread; comparing against a pet's own baseline over time turned out to be the more trustworthy signal.

10. Acknowledgments

Dataset

- "Thermal Image Dataset of cats for their health monitoring and classification." The ml-heat thermal-health model is trained and evaluated on this dataset (indoor/outdoor, healthy/sick, grouped by individual cat). All credit for the imagery belongs to the dataset's authors. The dataset is used here for research/educational purposes only.
- The classifier reproduces (and slightly beats, 76.5% vs 73.42%) the segmentation → temperature-histogram → logistic-regression baseline from the cited ml-heat paper that accompanies the dataset.

Open-source software

- App: [React Native](#), [React](#), [Expo](#) & [Expo Router](#), expo-camera, expo-constants, @react-native-async-storage/async-storage, @react-native-community/datetimepicker, [react-native-calendars](#), [react-native-chart-kit](#), [react-native-svg](#), @expo/vector-icons (Ionicons).
- Backends: [FastAPI](#), [Uvicorn](#), [Pydantic](#), [scikit-learn](#), [OpenCV](#), [NumPy](#), [pytesseract](#) + [Tesseract OCR](#), [joblib](#), python-multipart.
- Testing: [Jest](#), jest-expo, [@testing-library/react-native](#), [pytest](#), [httpx](#).